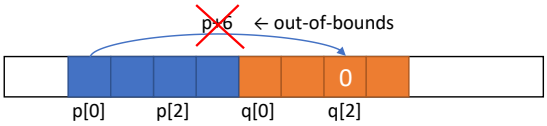

Fig. 2. In a memory model with data-flow provenance tracking, `p[6]` is not allowed to alias `q[2]`

2.3 Extending Provenance to Integers

The model we showed in the previous section does not support low-level language features like integer to pointer casts. To support this functionality, we can extend integers with provenance information: [1]

```
char *p = malloc(4);  // (val=0x10, obj=p)
char *q = (int*)0x10; // (val=0x10, obj=nil)

*q = 0; // UB, since obj=nil
if (p == q)
    *q = 1; // still UB; obj=nil

int v = (int)p;        // (val=0x10, obj=p)
int w = v + 2;         // (val=0x12, obj=p)

*(char*)w = 3;  // OK

char *r = malloc(4);   // (val=0x14, obj=r)
int x = v + (int)r;    // (val=0x24, obj=??)
int y = x - (int)r;    // (val=0x10, obj=??)
```

In this model, each integer and pointer variable tracks a numeric value plus the object it refers to, or `nil` if none. As in the previous model, addresses of objects, even if stored as integer variables, need to be derived from an object. Hence, the stores through `q` are UB. The accesses through `w` are well-defined since the value of this integer variable derives (data-flow wise) from a valid object. The last lines of the example show that provenance tracking breaks down when doing integer arithmetic operations. It is hard to assign meaningful semantics to cases like these.

A drawback of this model—fatal in practice—is that it blocks many integer optimizations, such as propagation of equalities as done by, e.g., global value numbering (GVN) or range analysis. For example, transforming "`(a == b) ? a : b`" into "`b`" is incorrect in this model: even if two integer variables compare equal, they may still have different provenances. We give a more complete example to demonstrate the problem:

```
char *p = malloc(4);  // (val=0x10, obj=p)
char *q = malloc(4);  // (val=0x14, obj=q)
int v = (int)p + 4;   // (val=0x14, obj=p)
int w = (int)q;       // (val=0x14, obj=q)

if (v == w)
    *(int*)w = 2;
```

In this program, `v` and `w` happen to have the same value, but they differ in their provenance. Hence it is not safe to replace "`*(int*)w = 2`" with "`*(int*)v = 2`". Doing so would introduce UB since "`v`" is only allowed to access object `p` and its offset is out-of-bounds. However, this sort

[1] We will use `int` to represent an integer type that is sufficiently large to hold a pointer for brevity.